

DeepRecall

How a structure-aware RAG turns your learning materials into a queryable, cited tutor

A learning & architecture guide

Document Cognition Layer - v0.1.0

1. The core idea

Most RAG fails because it treats documents as flat text. Documents have structure, hierarchy, and implicit relationships - and so should the system that reads them.

DeepRecall is a **Document Cognition Layer**. Instead of slicing your material into equal-sized blind chunks, it first *understands* the document - its sections, tables, code, warnings and how they relate - then retrieves and answers with that structure intact, always citing its sources.

Why this matters for learning materials

Textbooks, lecture notes, slide decks and lab manuals are **highly structured**: chapters, definitions, worked examples, step-by-step procedures, comparison tables, and cautionary notes. A naive RAG shreds that structure and gives you confident-sounding but ungrounded answers. DeepRecall keeps the structure, so studying becomes faster and more trustworthy.

Study problem	How DeepRecall helps
"Where was that defined?"	Definition blocks are detected as a chunk type and cited to the exact section.
"Give me the steps."	Procedures are recognised and boosted for how-to questions, returned in order.
"Which option is better?"	Comparison tables are preserved whole and boosted for decision questions.
"What are the gotchas?"	Warning/caution blocks are surfaced explicitly as caveats.
"Is this actually in my notes?"	If nothing relevant is found, it refuses instead of hallucinating.

The promise

Point DeepRecall at a folder of your course materials and ask questions in plain language. Every answer comes back grounded in, and cited to, the exact section of the exact file it came from - so you can trust it and go read the source.

2. How it works, end to end

Three stages turn raw study material into a grounded answer.

1. Ingestion

parse -> enrich -> chunk by meaning

2. Hybrid retrieval

4 indices -> RRF -> boost -> rerank

3. Generation

grounded, cited, confidence-scored

2.1 Ingestion - reading like a student, not a shredder

DeepRecall parses each file into a tree (Title -> Section -> Subsection -> Paragraph / List / Table / Code), enriches it with named entities and an *intent* per section, then chunks by **meaning** rather than token count.

Semantic chunk types it recognises:

Chunk type	What it captures in your notes
Definition	A term and its explanation.
Procedure	Numbered / how-to steps.
Comparison	A table weighing options.
Code example	Code paired with its surrounding explanation.
FAQ	A question and its answer.
Warning	A caution, gotcha or "never do this".

Why chunk by meaning?

A fixed 500-token window can split a definition from its term, or a step 3 from step 4. Semantic chunks keep each idea whole, so retrieval returns a complete, usable answer instead of a fragment.

2.2 Retrieval - four lenses, fused

The same question is asked of four complementary indices, then their rankings are merged. No single method wins every time, so DeepRecall combines their strengths.

Index	Strength	Good for
Dense (vectors)	Meaning / paraphrase	"explain in other words"
Sparse (BM25)	Exact keywords	specific terms, acronyms
Graph (entities)	Relationships	"what depends on X"
Structural	Hierarchy	parent section, siblings

The four ranked lists are merged with **Reciprocal Rank Fusion** (RRF), which combines rankings fairly even when their score scales differ wildly:

$$\text{score}(d) = \sum_i 1 / (k + \text{rank}_i(d)), \text{ with } k = 60$$

Then two finishing touches:

- **Intent-aware structural boost** - a how-to question lifts *procedure* chunks (+0.10); a troubleshooting question lifts *warning* chunks (+0.20); a decision question lifts *comparison* tables (+0.15).
- **Cross-encoder rerank** - the top candidates are re-scored by jointly reading the question and each chunk, for sharper ordering.

2.3 Generation - grounded, cited, honest

DeepRecall assembles the winning chunks (plus a parent-section summary and related siblings, de-duplicated) and produces an answer where every claim points back to a numbered source.

Feature	What it gives a learner
Source grounding	Every claim carries a [n] citation to a real section.
Confidence score	An honest signal of how well the corpus covered you.
"I don't know"	Refuses when nothing relevant is found - no bluffing.
Caveat surfacing	Warnings are pulled out so you don't miss them.

Standard RAG vs DeepRecall

Standard RAG: "Here's what I found..." | DeepRecall: "Based on 3 sources, the recommended approach is X, with caveat Y from [warning block]" - and it shows you exactly where X and Y came from.

3. A worked example

Imagine your study folder holds two notes: a Kubernetes operations guide and an API-gateway reference. You ask:

"How do I configure auto-scaling for the API gateway during high traffic?"

DeepRecall's trace

```
intent = how-to   entities = [API Gateway]
```

Rank	Chunk type	Section	Why it won
1	procedure	API Gateway 3.2	BM25 + dense + graph + how-to boost
2	procedure	Kubernetes 4.2	step list + how-to boost
3	code	Kubernetes 4.2	HPA YAML paired with its explanation

The generated answer stitches these into ordered guidance, then appends the caveat it found in section 4.4 - "never set minReplicas to 0 for a public gateway" - and lists all four sources. Confidence: 97%.

Honesty check

Ask it "How do I bake a sourdough loaf?" against the same folder and it refuses: no relevant sources. That refusal is a feature - it is the difference between a study aid you can trust and one that confidently misleads you the night before an exam.

4. Using it on your own materials

Three lines of Python turn a folder of notes into a tutor:

```
from deeprecall import DeepRecall

rag = DeepRecall()
rag.ingest_dir("my_course_notes") # .md / .txt / .html
print(rag.query("summarise chapter 4's key formulas").render())
```

Run the bundled demo

```
py examples/demo.py # full pipeline on a sample corpus
py -m pytest -q # 6 tests
```

Runs anywhere, upgrades anywhere

DeepRecall runs end-to-end with **zero heavy dependencies** (pure-Python fallbacks for embeddings, BM25, the graph, reranking and generation). Install optional backends and the same code auto-upgrades - no rewrites.

Install	Upgrades
sentence-transformers	Real semantic embeddings + cross-encoder rerank.
anthropic + API key	Fluent LLM answers (claude-opus-4-8) vs extractive.
chromadb / neo4j / elasticsearch	Production-scale vector / graph / keyword stores.
unstructured / layout models	PDF & Word parsing with visual structure.

Study-workflow ideas

- Drop a semester of lecture notes into one folder and quiz yourself with cited answers.
- Ask "what are the warnings/exceptions in chapter 6?" to build a gotchas list.
- Ask decision questions to auto-extract comparison tables for revision.
- Trust the "I don't know" - it tells you where your notes have gaps to fill.

Project: C:\Users\lemona\OneDrive\Desktop\DeepRecall - MIT licensed - built as an extensible reference implementation.